ABU-SSO — Unified Single Sign-On & Digital Campus Wallet



A unified authentication and campus services platform for **Ahmadu Bello University, Zaria**. One login grants access to **six independent campus services** plus a digital wallet with QR ID card.

Course: COEN838 — Advanced Software Engineering
Student: Idris Muhammad Abubakar (P25EGCP9003)

🌐 Live Deployment

Main Identity Provider

- **App:** https://abu-sso.onrender.com
- **Login:** https://abu-sso.onrender.com/login
- **Admin Panel:** https://abu-sso.onrender.com/admin
- **API Docs:** https://abu-sso.onrender.com/docs

Six Distributed SSO Services

Service	URL
Student Portal	abu-sso-portal.onrender.com
LMS (Moodle)	abu-sso-lms.onrender.com
Library	abu-sso-library.onrender.com
Hostel	abu-sso-hostel.onrender.com
Wi-Fi	abu-sso-wifi.onrender.com
Result Checker	abu-sso-results.onrender.com

> **Note:** Free tier spins down after 15 minutes of inactivity. First request after idle takes 30–60 seconds. Subsequent requests are fast.

Demo Credentials

Role	Matric / Staff No.	Password
Admin	`ADMIN/001`	`admin123`
Student	`P25EGCP9003`	`secret123`

✨ Features

- 🔐 **Single Sign-On** — JWT-based authentication for all ABU services
- 🎓 **Matric-Number Login** — students and staff use their official university number
- 🚫 **Bootstrap-Only Registration** — first user on a fresh deployment becomes admin; all subsequent accounts are created by administrators (matches real ABU ICT workflow)
- 🔑 **Secure Password Storage** — bcrypt hashing (cost ≥ 12)
- 🛡️ **Role-Based Access Control** — `student` / `staff` / `admin` enforced by middleware
- 🪪 **Digital ID Card** — QR code encoding matric number + role
- 💰 **Campus Wallet** — top-up, pay, transaction history
- 🌐 **Web Dashboard** — Jinja2 + vanilla JS, launchpad for six services
- 👑 **Admin Panel** — create users, change roles, activate/deactivate, top up wallets, view stats
- 🚀 **Six Distributed SSO Services** — Portal, LMS, Library, Hostel, Wi-Fi, Result Checker
- 📖 **Auto-Generated API Docs** — OpenAPI 3.0 at `/docs`
- ✅ **84 Automated Tests** — 4 layers (unit, service, integration, E2E)
- 🤖 **Continuous Integration** — GitHub Actions on every push
- 🛡️ **SAST Clean** — 0 findings (Bandit)
- ☁️ **Deployed** — Render (7 web services) + Neon (PostgreSQL)

🚀 Quick Start (Local)

```
# Clone
git clone https://github.com/idrismakr-png/abu-sso.git
cd abu-sso

# Create environment
conda create -n abu-sso python=3.12 -y
conda activate abu-sso

# Install
pip install -r requirements.txt

# Run the main app
uvicorn app.main:app --reload

Dashboard: http://127.0.0.1:8000/login

API docs: http://127.0.0.1:8000/docs

Run the Six Mock Services (optional)
In separate terminals:

cd mock-services/portal  && uvicorn main:app --reload --port 4001
cd mock-services/lms     && uvicorn main:app --reload --port 4002
cd mock-services/library && uvicorn main:app --reload --port 4003
cd mock-services/hostel  && uvicorn main:app --reload --port 4004
cd mock-services/wifi    && uvicorn main:app --reload --port 4005
cd mock-services/results && uvicorn main:app --reload --port 4006

The dashboard auto-detects whether you're on localhost or onrender.com and wires
the service cards accordingly.
```

🧪 Run Tests

```
# Fast tests (unit + service + integration)
pytest -q

# System tests (Playwright, real browser)
pytest tests/e2e -m e2e

# With coverage
pytest --cov=app --cov-report=term-missing
```

Expected: 79 fast tests + 5 E2E tests = 84 passing, 96% line coverage.

📚 Documentation

All engineering artefacts live in docs/:

Document	Description
SRS	Software Requirements Specification (Module I)
RTM	Requirements Traceability Matrix
Design	Architecture, data model, patterns (Module III)
Security	STRIDE threat model, SAST evidence (Module IV)
Technical Report	Engineering narrative (Modules I–V)
Test Results	Full test inventory, coverage report, SAST output
User Manual	End-user + installation guide
Risk & Maintenance	Risk register + maintenance plan (Module V)
Presentation Guide	Slides + demo script + defence Q&A
AI Use Declaration	Transparent disclosure (exam instruction F)

🏗️ Architecture

The Identity Provider (modular monolith, layered):

```
HTTP  →  Router  →  Service  →  Model  →  Database
        (thin)    (business)   (ORM)     (SQLite/PG)
```

Plus six distributed services that trust the IdP:



Each service verifies the token by calling the IdP's /auth/me — no shared session, no shared database.

See docs/DESIGN.md for full details.

Backend: FastAPI (Python 3.12)

Database: SQLite (dev) → PostgreSQL (prod, Neon)

Auth: JWT (HS256) + bcrypt

Authorization: RBAC middleware (require_role)

```
Testing: pytest + httpx + Playwright

CI: GitHub Actions

SAST: Bandit

Hosting: Render (7 services) + Neon (free tier)

📊 Project Stats
Metric  Value
API Endpoints (IdP) 17
Distributed SSO Services    6
Functional Requirements 19/19 implemented
Automated Tests 84 passing
Line Coverage   96%
SAST Findings   0
CI Runtime  ~50s (both jobs)
UML Diagrams    3

🔒 Roles & Access
Role    Permissions
student Own profile, own wallet, own ID card, dashboard, 6 SSO services
staff   Same as student
admin   All of the above + admin panel: create users, change roles,
activate/deactivate, top up any wallet, view system stats

Account creation:

First user on a fresh deployment → becomes admin (bootstrap).

All subsequent accounts → created by an admin via the Admin Panel.

Public registration is disabled after bootstrap. Students attempting to access
/admin/* receive HTTP 403 Forbidden.

🔐 Security Highlights
Passwords: bcrypt with auto-generated salt

Sessions: HS256 JWT, 15-minute expiry, ≥32-byte secret

Input validation: Pydantic schemas on every request

SQL injection: prevented by SQLAlchemy parameterisation

RBAC: enforced via require_role() dependency

Self-registration: disabled after bootstrap (bootstrap-only pattern)

SAST: Bandit clean (0 findings)

Secrets: .env gitignored; Render auto-generates in production

See docs/SECURITY.md for the full STRIDE analysis.
```

📄 License

MIT — see project documentation for details.

👤 Author

Idris Muhammad Abubakar
P25EGCP9003 — Department of Computer Engineering
Ahmadu Bello University, Zaria
📧 idrismakr@gmail.com

🙏 Acknowledgements

Built with FastAPI, SQLAlchemy, Neon, and Render.